

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

05

객체 지향 프로그래밍



05-5 상속

05-6 생성자

05-5 상속

- **상속(inheritance)**
  - 클래스 상속은 **extends** 키워드 사용
    - Dog 클래스는 Animal 클래스를 상속

Do it! 코딩해 보세요

```
class Animal {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

class Dog extends Animal {  // Animal 클래스를 상속한다.
}

public class Sample {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.setName("poppy");
        System.out.println(dog.name);
    }
}
```

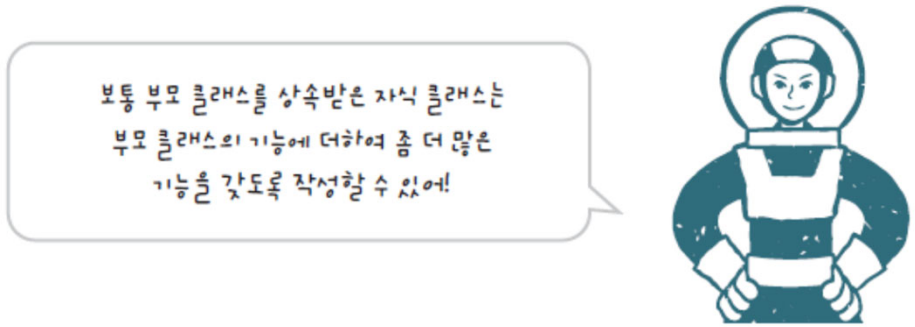
실행 결과

```
poppy
```

05-5 상속

▪ 자식 클래스의 기능 확장하기

- sleep 메서드 추가



Do it! 코딩해 보세요

```
class Animal {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

class Dog extends Animal {
    void sleep() {
        System.out.println(this.name+" zzz");
    }
}

public class Sample {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.setName("poppy");
        System.out.println(dog.name);
        dog.sleep();
    }
}
```

실행 결과

```
poppy
poppy zzz
```

05-5 상속

IS-A 관계란?

- 상속 관계
- Dog 클래스는 Animal 클래스를 상속하기 때문에 '개(Dog)는 동물(Animal)이다'라고 표현할 수 있는 관계
- IS-A 관계에 있을 때 자식 클래스의 객체는 부모 클래스의 자료형인 것처럼 사용할 수 있음

```
Animal dog = new Dog();  // Dog is a Animal
```

- 반대의 경우에는 컴파일 오류 발생

```
Dog dog = new Animal();  // 컴파일 오류 발생
```

05-5 상속

▪ 메서드 오버라이딩(method overriding)

- 부모 클래스의 메서드를 자식 클래스가 동일한 형태로 또다시 구현하는 행위(덮어쓰기)

Do it! 코딩해 보세요

```
class Animal {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

class Dog extends Animal {
    void sleep() {
        System.out.println(this.name + " zzz");
    }
}
```

```
class HouseDog extends Dog {
    void sleep() {
        System.out.println(this.name + " zzz in house");
    }
}


public class Sample {
    public static void main(String[] args) {
        HouseDog houseDog = new HouseDog();
        houseDog.setName("happy");
        houseDog.sleep();  // happy zzz in house 출력
    }
}
```

실행 결과

```
happy zzz in house
```

05-5 상속

▪ 메서드 오버로딩(method overloading)

- 메서드의 입력 항목이 다른 경우 동일한 이름의 메서드를 만드는 것

Do it! 코딩해 보세요

```
class Animal {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

class Dog extends Animal {
    void sleep() {
        System.out.println(this.name + " zzz");
    }
}
```

```
class HouseDog extends Dog {
    void sleep() {
        System.out.println(this.name + " zzz in house");
    }

    void sleep(int hour) {
        System.out.println(this.name + " zzz in house for " + hour + " hours");
    }
}

public class Sample {
    public static void main(String[] args) {
        HouseDog houseDog = new HouseDog();
        houseDog.setName("happy");
        houseDog.sleep();  // happy zzz in house 출력
        houseDog.sleep(3);  // happy zzz in house for 3 hours 출력
    }
}
```

실행 결과

```
happy zzz in house
happy zzz in house for 3 hours
```

05-5 상속

▪ 다중 상속이란?

- 클래스가 동시에 하나 이상의 클래스를 상속받는 것
- 자바는 다중 상속을 지원하지 않음
  - 불명확한 부분을 애초에 없앤 언어

이 코드는 실제 동작하지 않음 →

Do it! 코딩해 보세요

```
class A {
    public void msg() {
        System.out.println("A message");
    }
}

class B {
    public void msg() {
        System.out.println("B message");
    }
}

class C extends A, B {
    public void static main(String[] args) {
        C test = new C();
        test.msg();
    }
}
```

05-6 생성자

- **생성자(constructor)**
  - 메서드명이 클래스명과 동일하고 리턴 자료형을 정의하지 않는 메서드
  - 생성자 규칙
    - 클래스명과 매서드명이 같다.
    - 리턴 타입을 정의하지 않는다(void도 사용하지 않는다).
  - 생성자는 객체가 생성될 때 호출됨(new 키워드 사용)

```
new 클래스명(입력인수, …)
```

    - 객체 생성 방법이 생성자의 규칙과 맞지 않으면 컴파일 오류 발생

05-6 생성자

▪ 디폴트 생성자(default constructor)

- 생성자의 입력 항목이 없고 생성자 내부에 아무 내용이 없는 생성자
- 클래스에 생성자가 하나도 없다면 컴파일러는 자동으로 이와 같은 디폴트 생성자를 추가함
- 사용자가 작성한 생성자가 하나라도 구현되어 있다면 컴파일러는 디폴트 생성자를 추가하지 않음

Do it! 코딩해 보세요

```
(... 생략 ...)

class Dog extends Animal {
    Dog(){
    }

    void sleep() {
        System.out.println(this.name + " zzz");
    }
}

(... 생략 ...)
```

05-6 생성자

▪ 생성자 오버로딩(constructor overloading)

- 입력 항목이 다른 생성자를 여러 개 만들 수 있음

Do it! 코딩해 보세요

```
class Animal {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

class Dog extends Animal {
    void sleep() {
        System.out.println(this.name + " zzz");
    }
}
```

```
class HouseDog extends Dog {
    HouseDog(String name) {
        this.setName(name);
    }
    HouseDog(int type) {
        if (type == 1) {
            this.setName("yorkshire");
        } else if (type == 2) {
            this.setName("bulldog");
        }
    }

    void sleep() {
        System.out.println(this.name + " zzz in house");
    }
```

05-6 생성자

▪ 생성자 오버로딩(constructor overloading)

- 입력 항목이 다른 생성자를 여러 개 만들 수 있음

```
    void sleep(int hour) {
        System.out.println(this.name + " zzz in house for " + hour + " hours");
    }
}

public class Sample {
    public static void main(String[] args) {
        HouseDog happy = new HouseDog("happy");
        HouseDog yorkshire = new HouseDog(1);
        System.out.println(happy.name);  //happy 출력
        System.out.println(yorkshire.name);  //yorkshire 출력
    }
}
```

실행 결과

```
happy
yorkshire
```



감사합니다.